

Dissecting King of the Hill Strategy: A Comparative Study of Neural Network vs Heuristic Evaluation in the Early Middlegame

Hardik Shrestha

Abstract—This study investigates the effectiveness of neural networks versus heuristic-based linear models in evaluating positions from King of the Hill chess, a variant where kings can win by occupying central squares. The focus is on early middlegame positions (plies 20–40) where strategic king centralization decisions emerge. Two approaches are compared: a logistic regression model using handcrafted features that approximate traditional chess heuristics, and a multi-layer perceptron neural network using the same feature set. The features capture material balance, king centralization, central control, and piece mobility. Both models are trained to predict the eventual game winner from single board positions using maximum likelihood estimation with cross-entropy loss. Evaluation will use accuracy, precision, recall, F1-score, and ROC-AUC, with additional analysis comparing performance on positions with centralized versus peripheral kings. The project aims to test whether a modest neural network can better capture the non-linear strategic patterns introduced by King of the Hill’s central occupation win condition than a regularized linear baseline.

I. INTRODUCTION

Chess evaluation functions are a classic problem in artificial intelligence, combining pattern recognition, planning, and position assessment. King of the Hill (KotH), a popular chess variant on platforms such as Lichess, modifies the standard win condition: a player can also win by moving the king to one of four central squares (d4, e4, d5, e5). This rule directly changes how king safety and centralization should be evaluated.

In classical chess, the king usually remains sheltered during the opening and middlegame, with centralization reserved for simplified endgames. KotH inverts this principle by rewarding aggressive king advances toward the center. Players must trade off traditional king safety against opportunities for immediate central wins, creating non-linear strategic patterns that standard heuristics may not capture well.

This project studies whether a neural network can better model KotH-specific patterns than a linear evaluation function. The task is to predict, from a single position between plies 20 and 40, whether the side to move eventually wins. I compare two supervised learning approaches trained on the same handcrafted features: (1) logistic regression with L2 regularization as a linear baseline and (2) a feed-forward neural network that can represent non-linear feature interactions.

The main contributions are: (1) a feature-based dataset of early middlegame KotH positions extracted from online

games; (2) implementation and hyperparameter tuning of logistic regression and neural network baselines on a shared feature representation; and (3) an evaluation protocol that compares models both overall and on subsets defined by king centralization. Section II reviews related work on chess evaluation and variants. Section III details the data, features, and model training setup. Results and conclusions will be added in the final report.

II. RELATED WORK

Early chess engines relied on handcrafted evaluation functions built from material counts and positional terms combined linearly. These approaches, used in classical engines and described in overviews of computer chess, assume a traditional model of king safety and phase-dependent centralization.[1], [7]

Neural-network-based evaluation became prominent with systems that learn value and policy functions from self-play, most notably AlphaZero, which achieved superhuman performance in chess, shogi, and Go using a unified reinforcement learning algorithm.[2] Subsequent work and surveys describe how neural networks, including efficiently updatable architectures such as NNUE, have been integrated into top engines like Stockfish.[3], [4]

Work on chess variants, including Crazyhouse and related games, suggests that evaluation benefits from either variant-specific features or models with enough capacity to learn variant patterns from data. For example, CrazyAra uses deep neural networks to reach above world-champion level in Crazyhouse,[5] and earlier work learned piece values for Crazyhouse, suicide, and atomic chess from self-play.[6] These results support the idea that changed win conditions require either specialized heuristics or more flexible learned evaluation functions. To my knowledge, King of the Hill has not yet been studied in the same depth, which motivates this project.

III. METHODOLOGY

The task is framed as supervised binary classification: given a single KotH position, predict whether the side to move eventually wins the game.

A. Data Collection and Labeling

Game data are drawn from a public Lichess King of the Hill dataset using the Hugging Face datasets library (`lichess/king-of-the-hill-chess-games`), with

a cap of `max_games = 500` for tractability. For each decisive game ("1-0" or "0-1"), I reconstruct the game and extract positions between plies 20 and 40, corresponding to the early middlegame once opening theory has typically ended.

Each extracted position is stored as a FEN string along with the game identifier, ply index, side to move, and final result. The label is defined at the position level: it is 1 if the side to move eventually wins the game and 0 otherwise. Drawn games are excluded so that the target is a clear binary outcome.

B. Feature Engineering

Features are computed from each FEN position using the `python-chess` library. The final feature vector has 37 dimensions grouped into four categories:

- **Material:** Piece counts for each side (pawns, knights, bishops, rooks, queens), per-piece imbalances (white minus black), total material for each side based on standard piece values, and overall material imbalance.
- **King centralization:** For each king, distance to the board center, indicators for being on or near a central KotH square, and a simple safety count of nearby friendly pieces. I also include the difference in king distance between the two sides.
- **Central control and pawns:** Counts of how many times each side attacks the four central KotH squares, plus their difference; counts of pawns on central files for each side and their difference.
- **Mobility and turn:** Legal move counts for each side (computed by toggling the turn in the board object), their difference, and a binary feature indicating whether it is White to move.

All features are standardized using `StandardScaler` from `scikit-learn`. The scaler is fit on the training set and then applied to validation and test sets to avoid information leakage.

C. Train/Validation/Test Split

To avoid leaking information across splits, I split at the game level rather than the position level. A helper function first collects unique game identifiers, shuffles them with `random_state = 42`, and then assigns approximately 60% of games to the training set, 20% to the validation set, and 20% to the test set. All positions from a single game belong to the same split.

After splitting, I obtain $(X_{\text{train}}, y_{\text{train}})$, $(X_{\text{val}}, y_{\text{val}})$, and $(X_{\text{test}}, y_{\text{test}})$ and then apply the fitted scaler to produce standardized feature matrices for all three splits.

D. Logistic Regression Baseline

The first model is a regularized logistic regression implemented with `scikit-learn`'s `LogisticRegression`. It represents a linear evaluation function over the feature space. The configuration is: L2 penalty, `lbfgs` solver, `max_iter = 1000`, and `class_weight = "balanced"` to compensate for any label imbalance.

I tune the inverse regularization strength $C \in \{0.001, 0.01, 0.1, 1.0, 10.0\}$ by fitting on X_{train} and selecting the value with highest validation accuracy. For the chosen C , I evaluate on X_{test} and record accuracy, precision, recall, F1-score, ROC-AUC, and the confusion matrix for later comparison.

E. Neural Network Model

The second model is a feed-forward neural network implemented with `scikit-learn`'s `MLPClassifier`. It uses the same standardized features as input, so differences in performance can be attributed to model capacity rather than feature design.

I search over hidden-layer configurations and regularization strengths with a small grid: hidden layer sizes (64, 32), (100), and (50, 25); L2 regularization alpha values 0.0001, 0.001, and 0.01. For each combination, I train an MLP with ReLU activations, the Adam optimizer, `max_iter = 100`, `random_state = 42`, `early_stopping = True`, `validation_fraction = 0.1`, and `n_iter_no_change = 10`. I select the model with the best validation accuracy and evaluate it on X_{test} , computing the same metrics as for logistic regression.

In the final report, I will compare both models on these metrics and also analyze performance on subsets of positions with centralized versus non-centralized kings to see whether the neural network gains more in the positions where KotH-specific strategy matters most.

ACKNOWLEDGMENT

I would like to thank the course instructor and teaching assistants for their guidance, and the maintainers of public King of the Hill game databases used in this project.

REFERENCES

- [1] D. Silver *et al.*, "Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm," *arXiv preprint arXiv:1712.01815*, 2017.
- [2] D. Silver *et al.*, "A General Reinforcement Learning Algorithm that Masters Chess, Shogi, and Go through Self-Play," *Science*, vol. 362, no. 6419, pp. 1140–1144, 2018.
- [3] D. Klein, *Neural Networks for Chess*. arXiv:2209.01506, 2022.
- [4] "Efficiently Updatable Neural Network (NNUE)," Chess Programming Wiki, 2024. [Online]. Available: <https://www.chessprogramming.org/NNUE>
- [5] J. Czech, J. Schaarschmidt, and T. Stolzenberg, "Learning to Play the Chess Variant Crazyhouse Above World Champion Level with Deep Neural Networks and Human Data," *Frontiers in Artificial Intelligence*, vol. 3, 2020.
- [6] S. Droste and J. Fürnkranz, "Learning the Piece Values for Three Chess Variants," in *Computers and Games*, LNCS, vol. 5131, Springer, 2008, pp. 119–130.
- [7] A. Maesumi, "Playing Chess with Limited Look Ahead," *arXiv preprint arXiv:2007.02130*, 2020.